AIAC-22.3

NAME: Siri Vempati

HTNO:2303A51309

BATCH:05

Task 1 – Bias Detection in AI Suggestions

Prompt:

Create a resume shortlisting algorithm to select the best candidate, most qualified candidate,that ensures diversity and fairness,prioritizing experience and top universities.

Code :

Ass-21.3 > Ass-22.3.py > shortlist_candidates

```python
from dataclasses import dataclass
from typing import List
@dataclass
class Candidate:
    id: int
    years_experience: int
    skills_match: float
    university_tier: int
    degree_level: str
    achievements_score: float
    group: str
    final_score: float = 0.0
def experience_score(years, relevance_score):
    if years <= 2:
        years_score = 40
    elif years <= 5:
        years_score = 70
    elif years <= 10:
        years_score = 90
    else:
        years_score = 100
    return (0.6 * years_score) + (0.4 * relevance_score)
def education_score(university_tier, degree_level):
    tier_map = {1: 100, 2: 80, 3: 60, 4: 40}
    degree_map = {"phd": 100, "masters": 85, "bachelors": 70}
    return (0.7 * tier_map.get(university_tier, 40) +
            0.3 * degree_map.get(degree_level.lower(), 70))
def compute_final_score(candidate):
    exp_score = experience_score(candidate.years_experience,
                                 candidate.skills_match)
    edu_score = education_score(candidate.university_tier,
                                candidate.degree_level)
    return (
        0.40 * exp_score +
        0.25 * candidate.skills_match +
        0.20 * edu_score +
        0.10 * candidate.achievements_score +
```

```python
Ass-21.3 > Ass-22.3.py > shortlist_candidates
def compute_final_score(candidate):
        0.05 * 100
    )
def get_candidates_from_user():
    candidates = []

    n = int(input("Enter number of candidates: "))

    for i in range(n):
        print(f"\n--- Candidate {i+1} ---")

        years = int(input("Years of experience: "))
        skills = float(input("Skills match (0-100): "))
        tier = int(input("University tier (1-4): "))
        degree = input("Degree (bachelors/masters/phd): ")
        achievements = float(input("Achievements score (0-100): "))
        group = input("Group (for diversity, e.g., A/B): ")

        candidate = Candidate(
            id=i+1,
            years_experience=years,
            skills_match=skills,
            university_tier=tier,
            degree_level=degree,
            achievements_score=achievements,
            group=group
        )

        candidates.append(candidate)

    return candidates
def shortlist_candidates(candidates: List[Candidate], top_k: int, group_min_ratio: dict):

    for c in candidates:
        c.final_score = compute_final_score(c)

    candidates_sorted = sorted(candidates,
```

Output:

```
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL_LAB\AI_ASSISTANT_CODING> & C:/Users/vempa/AppData/Local/Programs/Python/Python311/python.exe c:/U
sers/vempa/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3/Ass-22.3.py
Enter number of candidates: 2

--- Candidate 1 ---
Years of experience: 4
Skills match (0-100): 80
University tier (1-4): 3
Degree (bachelors/masters/phd): phd
Achievements score (0-100): 80
Group (for diversity, e.g., A/B): a

--- Candidate 2 ---
Years of experience: 4
Skills match (0-100): 80
University tier (1-4): 3
Degree (bachelors/masters/phd): phd
Achievements score (0-100): 90
Group (for diversity, e.g., A/B): b

How many candidates to shortlist? 1

Enter diversity constraints:
Group name (e.g., A): a
Minimum ratio (e.g., 0.4): 0.8
Second group name (e.g., B): b
Minimum ratio (e.g., 0.4): 0.8

✅ Shortlisted Candidates:

ID: 2, Score: 78.0, Group: b
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL_LAB\AI_ASSISTANT_CODING>
```

Explanation:

The algorithm works by first taking input details for each candidate, such as their years of experience, skills match, university tier, degree, achievements, and group for diversity. It then calculates a weighted score where experience has the highest importance, followed by skills, education (giving preference to top universities), and achievements. After scoring, all candidates are ranked from highest to lowest. Finally, a fairness step is applied to ensure that the shortlisted candidates meet minimum diversity requirements across different groups, without ignoring merit. The result is a balanced shortlist that selects the most qualified candidates while also maintaining fairness and representation.

Task 2 – Privacy and Sensitive Data Handling

Prompt:

Write a program to store and process student records including roll number, marks, and email.

Code :

```python
class Student:
    def __init__(self, roll_no, marks, email):
        self.roll_no = roll_no
        self.marks = marks
        self.email = email

    def display(self):
        print(f"Roll No: {self.roll_no}, Marks: {self.marks}, Email: {self.email}")
def calculate_average(students):
    total = sum(s.marks for s in students)
    return total / len(students) if students else 0
students = []
n = int(input("Enter number of students: "))
for i in range(n):
    print(f"\nEnter details for student {i+1}:")
    roll = int(input("Roll Number: "))
    marks = float(input("Marks: "))
    email = input("Email: ")
    student = Student(roll, marks, email)
    students.append(student)
print("\nStudent Records:")
for s in students:
    s.display()
avg = calculate_average(students)
print(f"\nAverage Marks: {avg:.2f}")
top_student = max(students, key=lambda s: s.marks)
print("\nTop Scorer:")
top_student.display()
```

Output:

```
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL_LAB\AI_ASSISTANT_CODING> & C:
sers/vempa/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3/Ass
Enter number of students: 2

Enter details for student 1:
Roll Number: 1309
Marks: 90
Email: 2303A51309@sru.edu.in

Enter details for student 2:
Roll Number: 1279
Marks: 95
Email: 2303A51279@sru.edu.in

Student Records:
Roll No: 1309, Marks: 90.0, Email: 2303A51309@sru.edu.in
Roll No: 1279, Marks: 95.0, Email: 2303A51279@sru.edu.in

Average Marks: 92.50

Top Scorer:
Roll No: 1279, Marks: 95.0, Email: 2303A51279@sru.edu.in
```

Explanation

This program defines a **Student class** to store roll number, marks, and email for each student. It takes input for multiple students and stores them in a list. Then it processes the data by displaying all records, calculating the **average marks**, and identifying the **top-scoring student**. It demonstrates basic concepts like classes, lists, loops, and functions in Python.

Task 3 – AI-Generated Security Risks

Prompt:

Write a payment processing script for an online shopping system that handles card details and transactions.

Code

```
3 > Ass-22.3.py > payment
class Payment:
    def __init__(self, card_number, card_holder, expiry, cvv, amount):
        self.card_holder = card_holder
        self.expiry = expiry
        self.cvv = cvv
        self.amount = amount
    def validate_card_number(self):
        return len(self.card_number) == 16 and self.card_number.isdigit()
    def validate_expiry(self):
        try:
            exp_date = datetime.strptime(self.expiry, "%m/%y")
            return exp_date > datetime.now()
        except:
            return False
    def validate_cvv(self):
        return len(self.cvv) == 3 and self.cvv.isdigit()
    def mask_card(self):
        return "XXXX-XXXX-XXXX-" + self.card_number[-4:]
    def process_payment(self):
        if not self.validate_card_number():
            return "Invalid card number"

        if not self.validate_expiry():
            return "Card expired or invalid expiry date"

        if not self.validate_cvv():
            return "Invalid CVV"

        return f"Payment of ₹{self.amount} successful using {self.mask_card()}"
card_number = input("Enter card number (16 digits): ")
card_holder = input("Enter card holder name: ")
expiry = input("Enter expiry (MM/YY): ")
cvv = input("Enter CVV: ")
amount = float(input("Enter amount: "))
payment = Payment(card_number, card_holder, expiry, cvv, amount)
result = payment.process_payment()
print("\nTransaction Status:", result)
```

Output:

```
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL_LAB\AI_ASSISTANT_CODING> & C:/Users/vempa/AppData/Local/Programs/Python/Python311/python.exe c:
sers/vempa/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3/Ass-22.3.py
Enter card number (16 digits): 1234567891213141
Enter card holder name: siri
Enter expiry (MM/YY): 01/28
Enter CVV: 141
Enter amount: 12000

Transaction Status: Payment of ₹12000.0 successful using XXXX-XXXX-XXXX-3141
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL_LAB\AI_ASSISTANT_CODING>
```

Explanation

This program simulates a **payment system** using a Payment class. It takes card details (number, expiry, CVV) and transaction amount as input. The program validates the card number (must be 16 digits), checks if the expiry date is valid and not expired, and ensures the CVV is correct. If all validations pass, the payment is marked successful and the card number is masked for security. Otherwise, it returns an appropriate error message.

Task 4 – Accountability in Human–AI Collaboration

Prompt:

Write code for an autonomous traffic signal control system that manages traffic flow based on vehicle density at intersections.

Code :

```
import time
class TrafficSignal:
    def __init__(self):
        self.roads = {
            "North": 0,
            "South": 0,
            "East": 0,
            "West": 0
        }
    def update_density(self):
        print("\nEnter vehicle count at each road:")
        for road in self.roads:
            self.roads[road] = int(input(f"{road}: "))
    def get_priority_road(self):
        return max(self.roads, key=self.roads.get)
    def calculate_green_time(self, vehicles):
        base_time = 5  # minimum seconds
        extra_time = vehicles * 1  # 1 sec per vehicle
        return base_time + extra_time
    def run_cycle(self):
        self.update_density()
        priority = self.get_priority_road()
        vehicles = self.roads[priority]
        green_time = self.calculate_green_time(vehicles)
        print(f"\n🚦 Green signal for {priority} road")
        print(f"Vehicles: {vehicles}, Green Time: {green_time} seconds")
        time.sleep(2)  # simulate delay (shortened)
        print(f"🔴 Switching signal...\n")
signal = TrafficSignal()
cycles = int(input("Enter number of cycles: "))
for i in range(cycles):
    print(f"\n=== Cycle {i+1} ===")
    signal.run_cycle()
```

Output:

```
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL_LAB\AI_ASSISTANT_CODING> & C:/Users/vempa/AppData/Loc
sers/vempa/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3/Ass-22.3.py
Enter number of cycles: 2

=== Cycle 1 ===

Enter vehicle count at each road:
North: 3
South: 5
East: 6
West: 7

🚦 Green signal for West road
Vehicles: 7, Green Time: 12 seconds
🔴 Switching signal...


=== Cycle 2 ===

Enter vehicle count at each road:
North: 4
South: 5
East: 6
West: 7

🚦 Green signal for West road
Vehicles: 7, Green Time: 12 seconds
🔴 Switching signal...
```

Explanation

This program simulates an **intelligent traffic signal system** using a TrafficSignal class. It takes vehicle counts (density) from four directions—North, South, East, and West. The system selects the road with the **highest vehicle density** and gives it the green signal. The green signal duration is dynamically calculated based on the number of vehicles, ensuring heavier traffic gets more time. This process repeats for multiple cycles, improving traffic flow efficiency compared to fixed-timer signals.

Task 5 – Limitations of AI in Critical Systems

Prompt:

Write a program for an aircraft maintenance fault detection system that identifies possible issues based on sensor data.

Code:

```python
class Aircraft:
    def __init__(self, engine_temp, oil_pressure, vibration, fuel_level):
        self.engine_temp = engine_temp
        self.oil_pressure = oil_pressure
        self.vibration = vibration
        self.fuel_level = fuel_level
    def detect_faults(self):
        faults = []
        if self.engine_temp > 100:
            faults.append("High Engine Temperature")
        if self.oil_pressure < 30:
            faults.append("Low Oil Pressure")
        if self.vibration > 5:
            faults.append("Excessive Vibration")
        if self.fuel_level < 15:
            faults.append("Low Fuel Level")

        return faults
print("Enter Aircraft Sensor Data:")

engine_temp = float(input("Engine Temperature (°C): "))
oil_pressure = float(input("Oil Pressure: "))
vibration = float(input("Vibration Level: "))
fuel_level = float(input("Fuel Level (%): "))
aircraft = Aircraft(engine_temp, oil_pressure, vibration, fuel_level)
faults = aircraft.detect_faults()
print("\n🔍 Fault Detection Result:")
if faults:
    for f in faults:
        print("⚠️", f)
else:
    print("✅ No faults detected. Aircraft is in good condition.")
```

Output:

```
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL_LAB\AI_ASSISTANT_CODING> & C:/Users/vempa/AppData/Local
sers/vempa/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3/Ass-22.3.py
Enter Aircraft Sensor Data:
Engine Temperature (°C): 32
Oil Pressure: 70
Vibration Level: 50
Fuel Level (%): 90

🔍 Fault Detection Result:
⚠️Excessive Vibration
```

Explanation

This program simulates an **aircraft fault detection system** using a class called Aircraft. It takes sensor inputs such as engine temperature, oil pressure, vibration level, and fuel level. Each parameter is checked against predefined safety thresholds. If any value exceeds safe limits (e.g., high temperature or low oil pressure), the system flags it as a fault. Finally, it displays all detected issues, helping in **early identification of maintenance problems** and improving safety.

Task 6 – Ethical Use of AI-Generated Code

Prompt:

Write a program to manage student records for an academic assignment, including storing, updating, and displaying data.

Code :

```
class Student:
    def display(self):
        print(f"Roll No: {self.roll_no}, Name: {self.name}, Marks: {self.marks}")
students = []
def add_student():
    roll = int(input("Enter Roll Number: "))
    name = input("Enter Name: ")
    marks = float(input("Enter Marks: "))
    students.append(Student(roll, name, marks))
    print("✅ Student added successfully!\n")
def update_student():
    roll = int(input("Enter Roll Number to update: "))
    for s in students:
        if s.roll_no == roll:
            s.name = input("Enter new name: ")
            s.marks = float(input("Enter new marks: "))
            print("✅ Record updated!\n")
            return
    print("❌ Student not found!\n")
def display_students():
    if not students:
        print("No records found.\n")
        return

    print("\n📋 Student Records:")
    for s in students:
        s.display()
    print()
while True:
    print("1. Add Student")
    print("2. Update Student")
    print("3. Display Students")
    print("4. Exit")
    choice = input("Enter choice: ")
    if choice == "1":
        add_student()
    elif choice == "2":
```

Output:

```
PS C:\Users\vempa\OneDrive\Pictures\Desktop\HCL_LAB\AI_ASSISTANT_CODING> & C:/Users/vempa/AppData/Local/Pr
sers/vempa/OneDrive/Pictures/Desktop/HCL_LAB/AI_ASSISTANT_CODING/Ass-21.3/Ass-22.3.py
1. Add Student
2. Update Student
3. Display Students
4. Exit
Enter choice: 1
Enter Roll Number: 1309
Enter Name: siri
Enter Marks: 80
✅ Student added successfully!

1. Add Student
2. Update Student
3. Display Students
4. Exit
Enter choice: 3

📋 Student Records:
Roll No: 1309, Name: siri, Marks: 80.0

1. Add Student
2. Update Student
3. Display Students
4. Exit
Enter choice:
```

Explanation

This program uses a **Student class** to store individual student details like roll number, name, and marks. It maintains a list of students and provides a **menu-driven interface** where the user can add new records, update existing ones using roll number, and display all student data. The program uses loops and functions to repeatedly perform operations until the user chooses to exit, making it a simple student record management system.